

CHAPTER 6 TREES

Trees

Definition 6.1

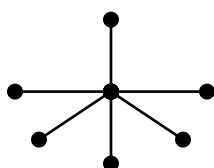
A graph G is **connected** if there is a path joining any two vertices of G .

A **tree** is a connected graph with no cycles.

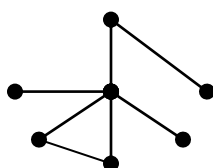
A **forest** is a disjoint collection of trees.

Example 6.1

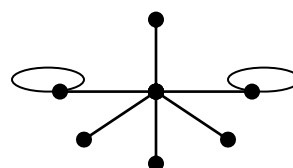
Determine which of the following graphs are trees.



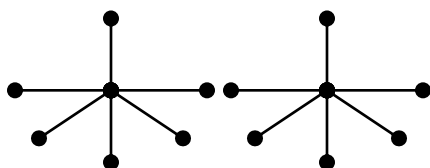
(i)



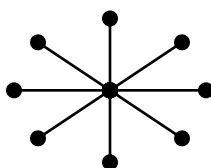
(ii)



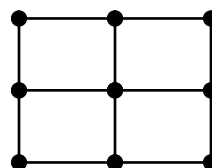
(iii)



(iv)



(v)



(vi)

(i) and (v) are connected graphs with no cycles, so they are trees.

There is a cycle in (ii), so it is not a tree.

There are loops (cycle of length 1) in (iii), so it is not a tree.

There is a cycle in (vi), so it is not a tree.

(iv) is not a tree because it is not connected. But it is a forest because it is a disjoint of two trees.

Theorem 6.1

Let n be the number of vertices of a graph G .

If G is a tree, then G has $n - 1$ edges.

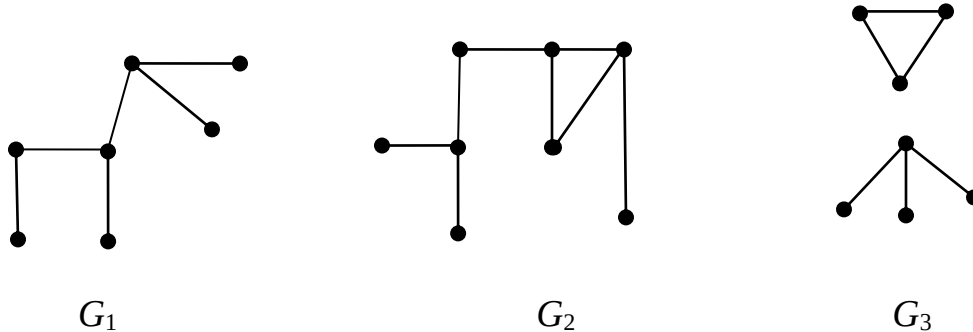
The contrapositive of Theorem 6.1

Let n be the number of vertices and let e be the number of edges of a graph G .

If $e \neq n - 1$, then G is not a tree.

However, if $e = n - 1$, G not necessarily to be a tree.
(The converse is not necessarily true.)

Example 6.2

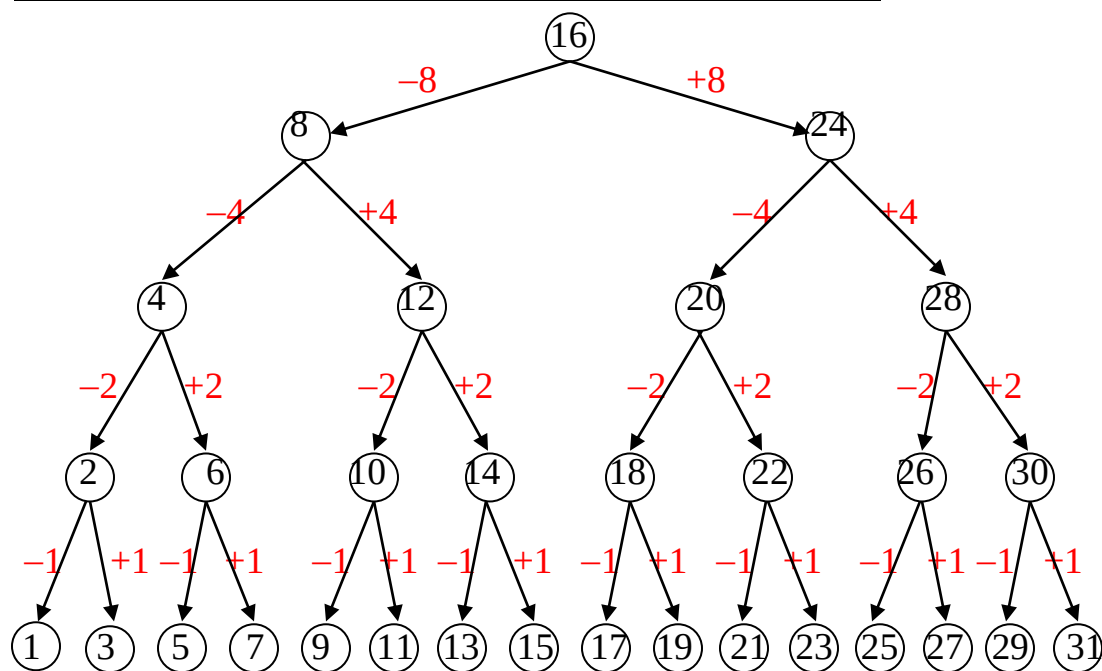


Tree support decision structures very well. We begin the discussion of this application with an example.

Example 6.3

There is a simple guessing game that goes like this: Someone thinks of a number between 1 and 31. You promise to guess the number correctly in a small number of guesses. You ask a question like, “Is the number x ?” Then the other person responds with “Yes”, “Less than x ,” “Greater than x .” Show that you can guarantee guessing the number in no more than five guesses.

The trick is always to guess the midpoint of the range of numbers remaining. Then even a wrong guess reduces the range by half, until finally only one number remain. The following figure shows how the guessing proceeds, beginning at 16. Each vertex in the tree is a decision point. Since there are only four levels in the tree, no more than five guesses will be needed.



The tree constructed in above figure is an example of rooted tree.

Rooted Tree

Definition 6.2

A **rooted tree** is a tree in which one vertex has been designated as the **root** v_0 and every edge is directed away from the root.

Notation: (T, v_0)

Theorem 6.2

Let (T, v_0) be a rooted tree. Then

- (a) there is no cycles in T .
- (b) v_0 is the only root of T .
- (c) Each vertex in T , other than v_0 , has in-degree one, and v_0 has in-degree zero.

Graphical representation of rooted trees

- (1) Draw the root v_0 . No edges enter v_0 but several may leave, and draw these edges downward. (In this case, the arrows indicating the directions of the edges in a rooted tree can be omitted, since the choice of root determines the directions of the edges.)
- (2) v_0 will be said at **level 0**. The terminal vertices of the edges beginning at v_0 will be called the **level 1** vertices.

- (3) v_0 is called the **parent** of these level 1 vertices, and the level 1 vertices are called the **offspring** (or **children**) of v_0 . Vertices with the same parent are called **siblings**.
- (4) The edges leaving a vertex of level 1 are drawn downward and terminate at various vertices, which are said to be at **level 2**. A parent-offspring relationship holds also for these levels.
- (5) The process above continues for as many levels as are required to complete the tree.
- (6) The largest level number of a tree is called the **height** of the tree.
- (7) The vertices of the tree that have no offspring are called the **leaves** (or **terminal vertices**) of the tree.
- (8) All vertices of T that can be reached by a path beginning at v are called the **descendants of v** .

```

graph TD
    v0((v0)) --> v1((v1))
    v0 --> v2((v2))
    v0 --> v3((v3))
    v1 --> v4((v4))
    v2 --> v5((v5))
    v2 --> v6((v6))
    v2 --> v7((v7))
    v3 --> v8((v8))
    v3 --> v9((v9))
    v3 --> v10((v10))
    v3 --> v11((v11))
    v4 --> v12((v12))
    v4 --> v13((v13))
    v4 --> v14((v14))
    v8 --> v15((v15))
    v8 --> v16((v16))
    v11 --> v17((v17))
  
```

Leaves are _____

UCCM1333 INTRODUCTORY DISCRETE MATHEMATICS

The height of the tree is _____

Example 6.5

Let $A = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}\}$ and

$T = \{(v_2, v_3), (v_2, v_1), (v_4, v_5), (v_4, v_6), (v_5, v_8), (v_6, v_7), (v_4, v_2), (v_7, v_9), (v_7, v_{10})\}$.

Show that T is a rooted tree and identify the root.

Vertex	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}
In-degree										

Example 6.6

Let $A = \{1, 2, 3, 4, 5, 6, 7, 8\}$ and T be the relation on A defined by $T = \{(1, 5), (1, 6), (2, 7), (2, 8), (3, 2), (3, 1), (3, 4)\}$. Determine whether T is a rooted tree.

Vertex	1	2	3	4	5	6	7	8
In-degree								

Trees as model

Example 6.7

The structure of an organization can be modeled using a rooted tree. Each vertex represents a position in the organization. An edge from one vertex to another indicates that the person represented by the terminal vertex is under the person represented by the initial vertex.

Theorem 6.3

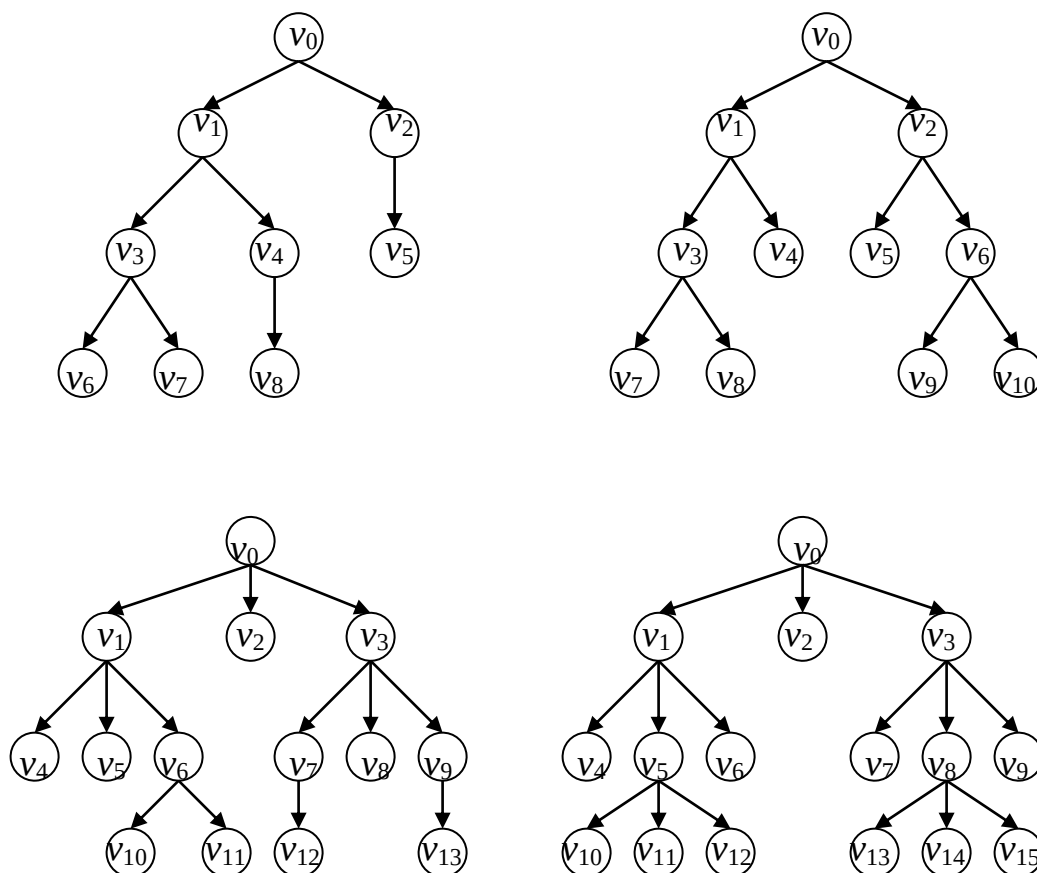
Let (T, v_0) be a rooted tree on a set A . Then

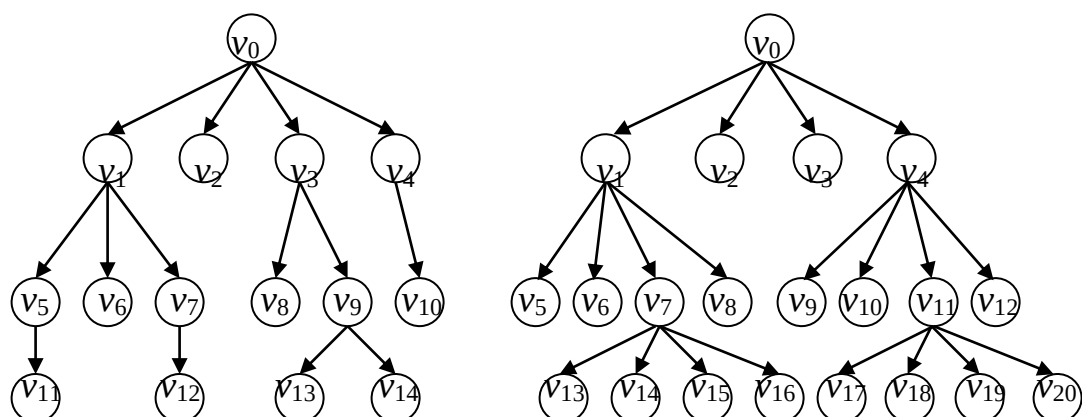
- (a) T is irreflexive.
- (b) T is asymmetric.
- (c) If $(x, y) \in T$ and $(y, z) \in T$, then $(x, z) \notin T$ for all x, y and z in A .

Definition 6.3

- (1) A tree T is called a **binary tree** if every vertex has at most two offspring.
- (2) If all vertices of T have exactly two offspring or no offspring, then T is called a **complete binary tree**.
- (3) In general, let $n \in \mathbb{Z}^+$.
A tree T is called an **n -tree** if every vertex has at most n offspring.
- (4) If all vertices of T have exactly n offspring or no offspring, then T is called a **complete n -tree**.

Example 6.8





Example 6.9

Let $A = \{1, 2, 3, 4, 5, 6, 7, 8\}$ and T be the relation on A defined by $T = \{(2, 5), (5, 3), (6, 2), (6, 4), (6, 8), (8, 1), (8, 7)\}$.

- Given that the digraph of T is a rooted tree. Draw the rooted tree.
- John uses 7 matches (Assume that all matches have equal length) to display the rooted tree in part (i) in such a way that each match is representing an edge in the rooted tree. He tries to remove one and only one match from the rooted tree and place it in a suitable place so that the new rooted tree is a binary tree with height 4. Describe all the possible solutions to John's problem.

Vertex	1	2	3	4	5	6	7	8
In-degree								

Labeled Trees

Definition 6.4

A **labeled tree** is a tree where each vertex is assigned a label.

In a labeled tree, we will represent the vertices simply as dots and show the label of each vertex next to the dot representing that vertex.

Example 6.10

Draw a labeled tree to represent the following fully parenthesized algebraic expression:

- (i) $(3 - (2 \times x)) + ((x - 2) - (3 + x))$
- (ii) $(3 \times (1 - x)) \div ((4 + (7 - (y + 2))) \times (7 + (x \div y)))$

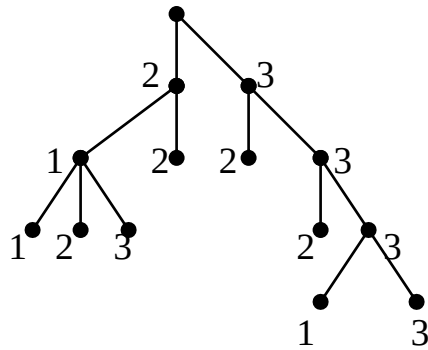
We assume, in such an expression, that no operation such as $+$, $-$, $\times$ or $\div$ can be performed until both of its arguments have been evaluated. Therefore, we cannot perform the central addition until we have evaluated $(3 - (2 \times x))$ and $((x - 2) - (3 + x))$. In this case, the central addition is called **central operator**. We cannot perform the central subtraction in $((x - 2) - (3 + x))$ until we evaluate $(x - 2)$ and $(3 + x)$. Here, the central subtraction is the central operator.

In this tree, the root is labeled with the central operator of the main expression. The two offspring of the root are labeled with the central operator of the expressions for the left and right arguments, respectively. If either argument is a constant or variable, instead of an expression, this constant or variable is used to label the corresponding offspring vertex. The process continues until the expression is exhausted.

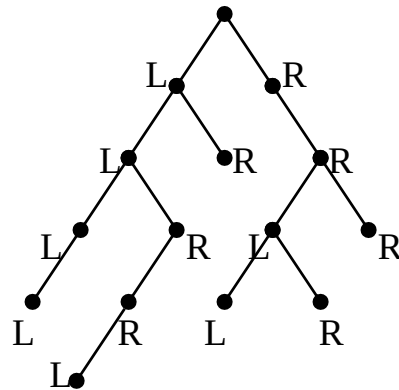
Positional tree

Consider an n -tree. Suppose the offspring of a vertex can be arranged in some order (ordered tree). If the vertices are labeled according to their position, we have a **positional tree**.

Example 6.11



Positional 3-tree



Positional binary tree

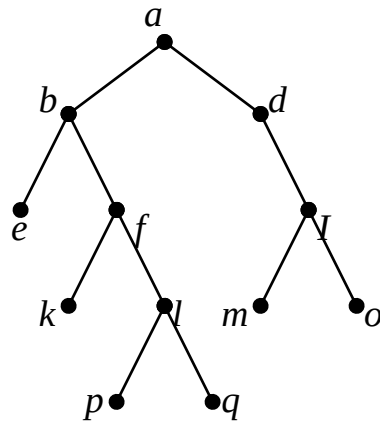
Theorem 6.4

If (T, v_0) is a rooted tree and $v \in T$, then $T(v)$ is also a rooted tree with root v . We will say that $T(v)$ is the **subtree** of T beginning at v .

Example 6.12

Consider the tree T of Example 6.5. Draw the subtrees $T(v_5)$, $T(v_2)$ and $T(v_6)$ of T .

Example 6.13



Tree T

a left subtree $T(b)$ of T

a right subtree $T(f)$ of T

Note:

In a binary positional tree, each vertex has two potential (not necessarily present) offspring, the first offspring v_L is called the **left offspring (left child)** and the second v_R is called the **right offspring (right child)**.

The vertex v_L together with all its descendants form a **left subtree $T(v_L)$** of T .

The vertex v_R together with all its descendants form a **right subtree $T(v_R)$** of T .

Trees Searching

Searching for items in a list is one of the most important tasks that arise in computer science. Our primary goal is to implement a searching algorithm that finds items efficiently when the items are totally ordered.

Performing a certain task (such as printing the label, compilation, etc.) at a vertex is described as **visiting** the vertex.

The process of visiting each vertex of a tree in some specified order will be called **searching** the tree or performing a **tree searching**.

(1) Algorithm PREORDER

Step 1

Visit the root v .

[Visit the root]

Step 2

If v_L exists, then apply this algorithm to $(T(v_L), v_L)$.

[Search the left subtree if it exists]

Step 3

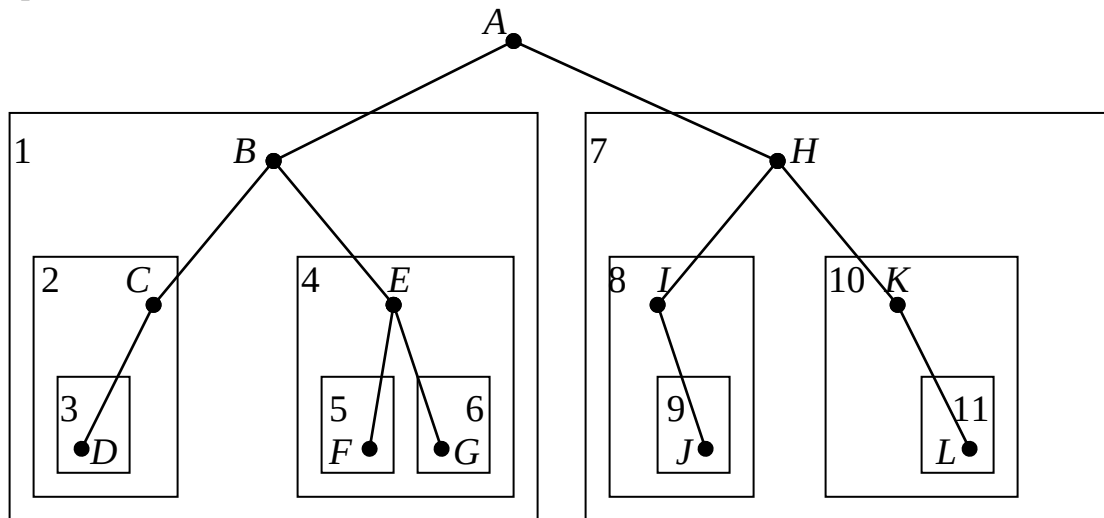
If v_R exists, then apply this algorithm to $(T(v_R), v_R)$.

[Search the right subtree if it exists]

End of Algorithm

Example 6.14

Consider the following binary tree. Suppose “visiting a vertex” means “printing its label”. By the pre-order search, what sequence of labels will be printed?



By PREORDER search, the vertices are visited in the sequence:

Example 6.15

Consider the fully parenthesized expression:

$$(a - b) \times (c + (d \div e))$$

Draw a binary tree and apply PREORDER search.

By PREORDER search, the vertices are visited in the sequence

This is called **prefix form**, or **Polish form** of the given algebraic expression.

The Polish form of an algebraic expression is interesting because it represents the expression unambiguously, without the need for parentheses.

UCCM1333 INTRODUCTORY DISCRETE MATHEMATICS

To evaluate an expression in Polish form:

- 1) Start searching from the left to right until we find a string of the form Fxy , where F is the symbol for a binary operation (say $+$, $-$, $\times$, $\div$, and so on) and x and y are numbers.
- 2) Evaluate xFy .
- 3) Continue this procedure until only one number remains.

Example 6.16

Evaluate $\times - 6 4 + 5 \div 2 2$.

Steps:

- 1) $\times - 6 4 + 5 \div 2 2$. The first string of the correct type is $- 6 4$. Replace this $(6 - 4)$ by 2.
- 2) $\times 2 + 5 \div 2 2$. Replace $\div 2 2$ ($2 \div 2$) by 1.
- 3) $\times 2 + 5 1$. Replace $+ 5 1$ ($5 + 1$) by 6.
- 4) $\times 2 6$. Replace $\times 2 6$ (2×6) by 12.
- 5) 12.

(2) Algorithm INORDER

Step 1

If v_L exists, then apply this algorithm to $(T(v_L), v_L)$.

[Search the left subtree if it exists]

Step 2

Visit the root v .

[Visit the root]

Step 3

If v_R exists, then apply this algorithm to $(T(v_R), v_R)$.

[Search the right subtree if it exists]

End of Algorithm

(3) Algorithm POSTORDER

Step 1

If v_L exists, then apply this algorithm to $(T(v_L), v_L)$.

[Search the left subtree if it exists]

Step 2

If v_R exists, then apply this algorithm to $(T(v_R), v_R)$.

[Search the right subtree if it exists]

Step 3

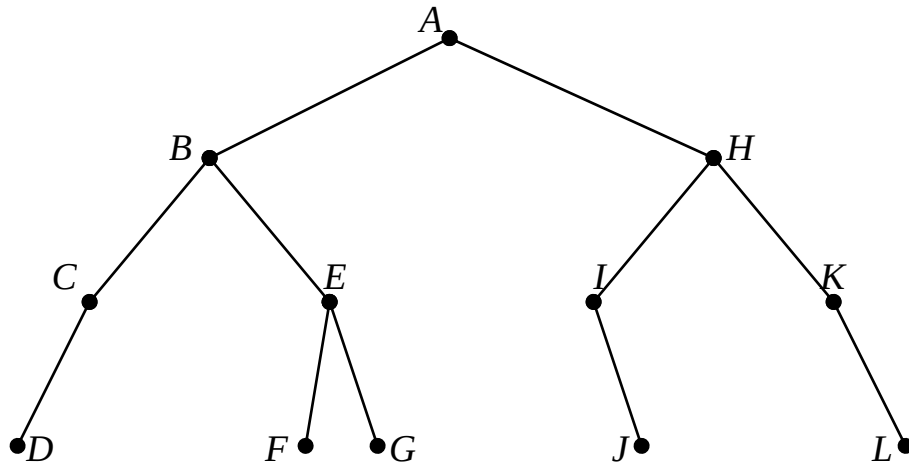
Visit the root v .

[Visit the root]

End of Algorithm

Example 6.17

Apply INORDER search and POSTORDER search to the tree of Example 6.14.

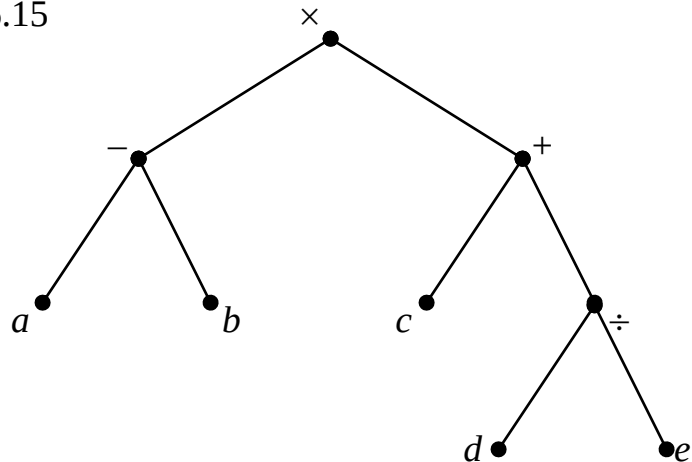


INORDER search:

POSTORDER search:

Example 6.18

Apply INORDER search and POSTORDER search to the tree of Example 6.15



- (i) By INORDER search, the vertices are visited in the sequence:

Note that this is exactly the expression that in Example 6.12 with all parentheses removed. Since the algebraic symbols lies between their arguments, this is often called **infix notation**.

The above expression is ambiguous without parentheses. It could have come from the expression $a - (b \times ((c + d) \div e))$, which would have produced a different tree. Thus the tree cannot be recovered

UCCM1333 INTRODUCTORY DISCRETE MATHEMATICS

from the output of search procedure INORDER. However the tree can be recovered from the Polish form produced by PREORDER. For this reason, Polish notation is often better for computer applications, although infix form is more familiar to human beings.

- (ii) By POSTORDER search, the vertices are visited in the sequence:

Since the algebraic symbols lie after their arguments, this is often called **postfix** or **reverse Polish form**.

(The postfix is evaluated in a manner similar to prefix, except that the operator symbol is after its arguments rather than before them.)

Example 6.19

Evaluate $2\ 1 - 3\ 4\ 2 \div + \times$.

- 1) The first string of the correct type is $2\ 1 -$. Replace this $(2 - 1)$ by 1.
- 2) $1\ 3\ 4\ 2 \div + \times$. Replace $4\ 2 \div (4 \div 2)$ by 2.
- 3) $1\ 3\ 2 + \times$. Replace $3\ 2 + (3 + 2)$ by 5.
- 4) $1\ 5 \times$. Replace $1\ 5 \times (1 \times 5)$ by 5.
- 5) 5.

Spanning Trees

Definition 6.5

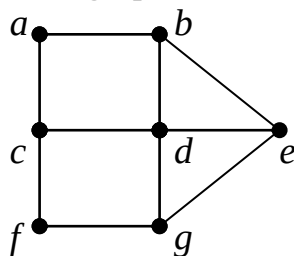
Suppose R is a symmetric relation represented by a connected graph G . Suppose some edges of R are removed until the resulting graph is a tree T .

T is called a **spanning tree** of R .

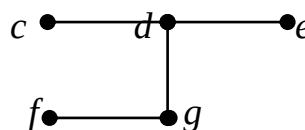
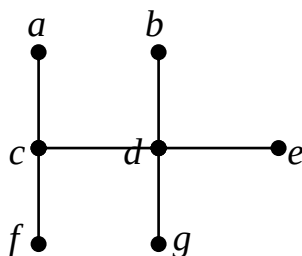
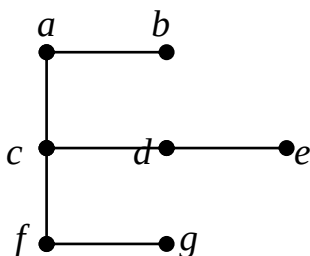
In other words, a spanning tree of a graph G is a subgraph of G that is a tree containing every vertex of G .

Example 6.20

Consider the graph G .



Which of the following are spanning trees of G ?



For any connected graph G , we can find a spanning tree systematically by using either of two methods:

Cutting-down method

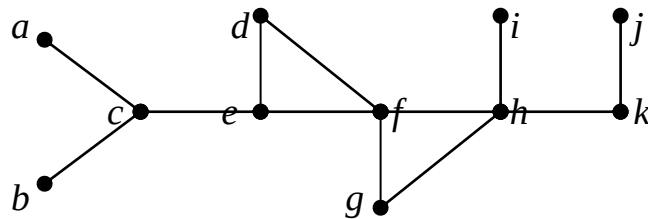
We start by choosing any cycle in G and removing one of its edges. (If there are no cycles, the graph G is itself a spanning tree) Since we cannot disconnect a graph by removing just one edge from a cycle, we still have a connected graph. We now repeat this procedure until there are no cycles left; this gives our spanning tree.

Building-up method

We select edges of G one at a time in such a way that no cycles are created, and repeat this procedure until all vertices are included.

Example 6.21

Use cutting down method and building up method to find a spanning tree for the graph below:



Minimal spanning trees

Definition 6.6

- 1) A **weighted graph** is graph for which each edge is labeled with a numerical value called its **weight**.
- 2) A **minimal spanning tree** in a connected weighted graph is a spanning tree that has the smallest possible sum of weight of its edges.

Prim's algorithm

Let R be a symmetric, connected relation with n vertices.

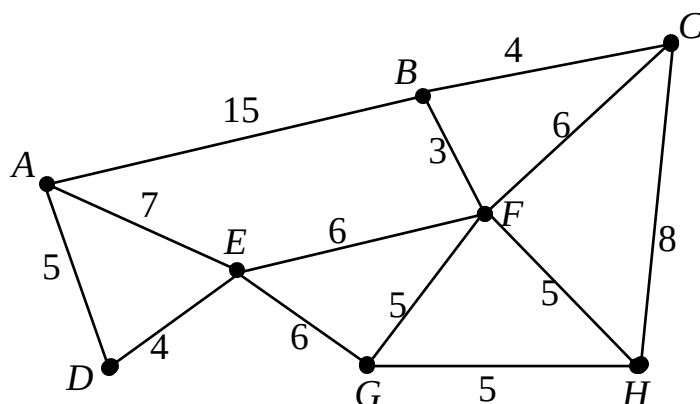
Steps:

1. Choose a vertex v_i of R . Let $V = \{v_i\}$ and $E = \{\}$.
2. Choose a nearest neighbor (with minimal weight) vertex v_j of V that is adjacent with v_i , $v_i \in V$, which the edge (v_i, v_j) does not form a cycle with members of E .
Add v_j to V and add (v_i, v_j) to E .
3. Repeat step 2 until $|E| = n - 1$. Then V contains all n vertices of R , and E contains the edges of a minimal spanning tree of R .

End algorithm.

Example 6.22

A communications company is investigating the costs of upgrading links between the relay stations it owns. The weighted graph in figure below shows the stations and the cost in millions of dollars for upgrading each link. Use Prim's algorithm to find a minimal spanning tree for the communication network.



Choose an arbitrary vertex, choose A.

Nearest neighbor of A with minimal weight: D. (A, D)

Nearest neighbor of A and D with minimal weight: E. (D, E)

Nearest neighbor of A, D and E with minimal weight: F. (E, F)

Nearest neighbor of A, D, E and F with minimal weight: B. (F, B)

Nearest neighbor of A, D, E, F and B with minimal weight: C. (B, C)

Nearest neighbor of A, D, E, F, B and C with minimal weight: G (F, G)

Thus the minimum spanning tree, $T = \{(A, D), (D, E), (E, F), (F, B), (B, C), (F, G)\}$ and the weight is 27.

Kruskal's algorithm

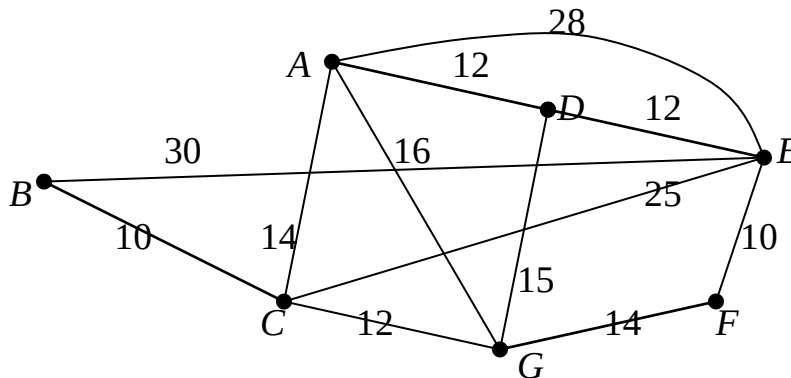
Let R be a symmetric, connected relation with n vertices and let $S = \{e_1, e_2, e_3, \dots, e_k\}$ be the set of weighted edges of R .

Steps:

1. Choose an edge e_1 in S of least weight. Let $E = \{e_1\}$. Replace S with $S - \{e_1\}$.
 2. Select an edge e_i in S of least weight that will not make a cycle with members of E . Replace E with $E \cup \{e_i\}$ and S with $S - \{e_i\}$.
 3. Repeat step 2 until $|E| = n - 1$.
- End algorithm.

Example 6.23

Use Kruskal's algorithm to find a minimal spanning tree for the relation given by the graph in figure below.



Since there are 7 vertices in the graph, a minimal spanning tree has only 6 edges.

- 1) Select the two edges (B, C) and (E, F) of minimum weight 10.
- 2) Select the three edges (A, D) , (C, G) and (D, E) of weight 12.
- 3) Select the edge (A, C) [or (G, F)] of weight 14.

Thus the minimal spanning tree, $T = \{(A, D), (A, C), (B, C), (C, G), (D, E), (E, F)\}$ and the weight is 70.